Bayesian Machine Learning report : Architecture and Inference Choices in Bayesian Neural Networks

Mohamed Aloulou
PSL University
mohamed.aloulou0@dauphine.eu

Lucas Mebille
PSL University
lucas.mebille@dauphine.eu

Abstract

This report reviews and critiques Sheinkman and Wade [2025a] on Bayesian neural networks (BNNs). We present the theoretical framework, contrasting mean-field variational inference (VI) and Hamiltonian Monte Carlo (HMC), and three model combination strategies: deep ensembles, pseudo-BMA, and stacking. The paper's main finding is that wide ReLU networks trained with VI offer the best trade-off between accuracy, calibration, and runtime, while HMC tends to underestimate uncertainty. We reproduce the width and depth experiments and propose two extensions: a Laplace prior for sparsity, and a low-rank variational family that captures limited posterior correlations. Our results show that prior choice has a stronger effect at large width than the original study suggests. For model averaging, stacking and deep ensembles consistently outperform pseudo-BMA, which collapses onto a single model. We conclude with limitations and directions for future work.

1 Bayesian Neural Networks: Setup and Notation

1.1 Bayesian Neural Networks

A Bayesian Neural Network (BNN) extends a classical neural network by placing probability distributions over the model parameters rather than learning a single output. In the formulation adopted in the paper, the neural network is defined as a hierarchical probabilistic model where the output is a random variable generated as

$$y \sim \mathcal{N}(b_{L+1} + W_{L+1} z_L, \sigma),$$

with hidden layer activations defined recursively as

$$z_l = g(b_l + W_l z_{l-1}), \quad l = 1, \ldots, L,$$

and input $z_0 = x$. Here W_l and b_l denote the weight matrices and biases of layer l, $g(\cdot)$ is a nonlinear activation function (ReLU or sigmoid in their experiments), and σ represents the noise scale. Instead of estimating fixed parameters, the Bayesian framework places prior distributions on the network parameters $\theta = \{W_l, b_l\}_{l=1}^{L+1}$.

Training then consists of estimating the posterior distribution $p(\theta \mid D)$, where D denotes the training dataset. This process is called *inference*. Note that this term is used differently in classical deep learning, where *inference* usually refers to making predictions after training. In Bayesian modeling, by contrast, inference means learning or approximating the posterior distribution over parameters from data.

Using Bayes rule, the posterior is $p(\theta \mid D) = \dfrac{p(D \mid \theta)p(\theta)}{\int p(D \mid \theta)p(\theta)\, d\theta}$.

The integral is taken over all parameters of the neural network. Since modern networks contain thousands or millions of parameters, this high-dimensional integral cannot be computed exactly. Therefore, the posterior is intractable in practice and must be approximated using methods such as variational inference or MCMC, which we discuss in subsection 1.2.

To fully specify the model, the authors introduce priors over the weights and biases.

$$b_l \sim \mathcal{N}\left(0, \frac{1}{4L}\right), \qquad W_1 \sim F\left(0, \frac{1}{4L}\right), \qquad W_l \sim F\left(0, \frac{4}{D_{l-1}}\right), \quad l = 2, \ldots, L+1,$$

In particular, the biases follow Gaussian priors while the weights follow whereas $F(\mu, \sigma^2)$ denotes either a Gaussian distribution or a Student-t distribution with five degrees of freedom. The paper motivates the use of Student-t priors by empirical observations that the weights of neural networks trained with SGD often exhibit heavy-tailed distributions. However, the specific choice of five degrees of freedom is not justified nor empirically investigated. Since heavy-tailed priors can significantly influence posterior behavior, especially in high-dimensional parameter spaces, the absence of a sensitivity analysis for this hyperparameter weakens the interpretability of the results.

Another design choice is the variance scaling of the weights, which is proportional to the inverse of the previous layer width D_{l-1}. This is close to classical weights initialization (such as in He et al. [2015]). While this scaling may keep the prior outputs well behaved, the paper does not justify why such a training heuristic should define the prior, making it unclear whether these variances encode meaningful prior beliefs or simply reproduce standard initialization practices.

Finally, observation noise standard deviation has the prior

$$\sigma \sim |\mathcal{N}(0, 0.001)|.$$

This is a half-normal prior with a very small scale. It forces the noise in the likelihood $y \sim \mathcal{N}(f(x;\theta), \sigma)$ to stay very small. As a result, most of the variability in the data must be explained by the neural network. This choice may help stabilize inference. However, the paper does not justify this value. It also does not test other variances.

1.2 Approximate Inference Objectives

Exact computation of the posterior $p(\theta \mid D)$ is infeasible for neural networks with more than a handful of parameters, because it involves a very high-dimensional integral. Therefore, the paper compares two scalable methods: mean-field variational inference (VI) and Hamiltonian Monte Carlo (HMC, using NUTS). The paper largely treats these inference methods as prerequisites and does not re-derive them in detail; here we only remind the key idea and how inference and prediction are done in practice.

1.2.1 Mean-field Variational Inference (VI)

Main assumption. Mean-field VI approximates the true posterior $p(\theta \mid D)$ by a simpler distribution $q_\lambda(\theta)$ that factorizes across parameters:

$$q_\lambda(\theta) = \prod_{i=1}^{P} \mathcal{N}(\theta_i; \mu_i, \sigma_i^2),$$

where P is the number of parameters, and $\lambda = (\mu_1, \ldots, \mu_P, \sigma_1, \ldots, \sigma_P)$ are free variational parameters. This assumption means that *all parameters are independent under* q_λ and each marginal is Gaussian.

Inference (training). Inference consists of choosing λ so that $q_\lambda(\theta)$ is close to $p(\theta \mid D)$. This is done by maximizing the evidence lower bound (ELBO):

$$\mathcal{L}(\lambda) = \mathbb{E}_{q_\lambda}\big[\log p(D \mid \theta)\big] - \mathrm{KL}\big(q_\lambda(\theta) \,\|\, p(\theta)\big).$$

In practice, the expectation is estimated with Monte Carlo samples of θ and optimized with stochastic gradients.

Prediction. After training, we have *two numbers per parameter* (a mean μ_i and a standard deviation σ_i). To make predictions, we sample several parameter vectors

$$\theta^{(s)} \sim q_\lambda(\theta), \quad s = 1, \ldots, S,$$

run the network forward for each sample, and average the resulting predictive distributions:

$$p(y \mid x, D) \approx \frac{1}{S} \sum_{s=1}^{S} p\big(y \mid x, \theta^{(s)}\big).$$

This Monte Carlo averaging is what converts the uncertainty in θ into uncertainty in predictions.

1.2.2 Hamiltonian Monte Carlo (HMC / NUTS)

Core idea. Unlike VI, HMC aims to sample θ *directly from the true posterior*:

$$\theta^{(s)} \sim p(\theta \mid D).$$

In the paper, this is done with the No-U-Turn Sampler (NUTS), a standard adaptive variant of HMC.

Inference (sampling). Inference consists of constructing a Markov chain whose stationary distribution is $p(\theta \mid D)$, and collecting samples after warmup:

$$\{\theta^{(1)}, \ldots, \theta^{(S)}\} \approx p(\theta \mid D).$$

This is typically much more computationally expensive than VI for neural networks.

Prediction. Prediction uses the posterior samples in the same Monte Carlo way as VI, but with samples coming from $p(\theta \mid D)$ instead of q_λ:

$$p(y \mid x, D) \approx \frac{1}{S} \sum_{s=1}^{S} p\big(y \mid x, \theta^{(s)}\big).$$

1.2.3 Comparison and theoretical trade-offs

Both VI and HMC produce uncertainty-aware predictions by Monte Carlo averaging

$$p(y \mid x, D) \approx \frac{1}{S} \sum_{s=1}^{S} p\big(y \mid x, \theta^{(s)}\big),$$

so the main difference is *how we obtain* the samples $\theta^{(s)}$: VI samples from an approximate posterior $q_\lambda(\theta)$, while HMC samples from (an asymptotically exact) posterior $p(\theta \mid D)$.

- **HMC (NUTS).** HMC targets the full posterior without imposing a parametric shape, so it can represent non-Gaussian features and dependencies between parameters. However, it is far more computationally expensive (each step needs gradient evaluations) and in very high-dimensional, multi-modal settings it may mix poorly, yielding overconfident or unreliable uncertainty estimates in practice.
- **Mean-field VI.** VI turns inference into an optimisation problem by maximising the ELBO. It is typically fast and works well with stochastic gradients, but it restricts the posterior to a factorised (diagonal) Gaussian family. This limitation can under-represent correlations between weights and can miss multi-modal or heavy-tailed posterior structure.

For these reasons, in our extension, we also consider a **low-rank variational family** Variational Inference, which keeps most of the scalability of mean-field VI while allowing a limited amount of correlation between parameters (a compromise between diagonal VI and full-rank VI). This extension is explained more in details in subsection 3.3.

2 Bayesian Model Averaging and Stacking

In the preceding section, we discussed how to perform inference for a *single* Bayesian neural network. However, the posterior landscape of neural networks is highly multimodal: different random initializations, architectures, or hyperparameter choices lead to distinct posterior modes that may yield qualitatively different predictions. This raises a fundamental question: *how should we aggregate predictions from multiple candidate models?*

Let $\{M_1, \ldots, M_K\}$ denote a set of K candidate models. Each model M_k defines a likelihood $p(y \mid x, \theta_k, M_k)$ and a prior $p(\theta_k \mid M_k)$, inducing a posterior $p(\theta_k \mid \mathcal{D}, M_k)$ and a predictive distribution

$$p(y \mid x, \mathcal{D}, M_k) = \int p(y \mid x, \theta_k, M_k)\, p(\theta_k \mid \mathcal{D}, M_k)\, \mathrm{d}\theta_k.$$

Our goal is to construct a *combined* predictive distribution of the form

$$p_{\mathrm{comb}}(y \mid x, \mathcal{D}) = \sum_{k=1}^{K} w_k\, p(y \mid x, \mathcal{D}, M_k), \tag{1}$$

where $\mathbf{w} = (w_1, \ldots, w_K)$ lies in the probability simplex $\mathcal{S}^{K-1} = \{\mathbf{w} \in [0,1]^K : \sum_{k=1}^K w_k = 1\}$. The key question is: *how should we choose the weights* $\mathbf{w}$*?*

The answer depends critically on the **M-closed vs. M-open** distinction:

- **M-closed setting**: The true data-generating process $p^*(y \mid x)$ is assumed to coincide with one of the candidate models M_{k^*} for some $k^* \in \{1, \ldots, K\}$. In this (idealized) setting, classical Bayesian Model Averaging (BMA) is optimal.
- **M-open setting**: The true process p^* lies *outside* the candidate set. This is the realistic scenario for deep learning, where no finite neural network exactly captures the data distribution. Here, BMA can be overconfident and alternative weighting schemes become necessary.

2.1 Classical Bayesian Model Averaging (BMA)

In the M-closed framework, Bayesian reasoning prescribes weighting each model by its posterior probability. Given a prior $p(M_k)$ over models, Bayes' theorem yields

$$p(M_k \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid M_k)\, p(M_k)}{\sum_{j=1}^{K} p(\mathcal{D} \mid M_j)\, p(M_j)}.$$

The BMA predictive distribution is then precisely Equation 1 with $w_k = p(M_k \mid \mathcal{D})$.

However, in M-open scenarios, BMA exhibits problematic behavior:

- Concentration on a single model: As $N \to \infty$, the posterior $p(M_k \mid \mathcal{D})$ concentrates on the model closest to p^* in KL divergence, effectively collapsing to a point estimate in model space.
- Computational burden: Computing marginal likelihoods for neural networks is intractable; approximations (e.g., the Laplace approximation or variational bounds) introduce additional errors.

To address these issues, the authors explore three alternative combination strategies.

2.2 Method 1: Deep Ensembles of BNNs

The simplest approach assigns uniform weights: $w_k = \frac{1}{K}, \quad k = 1, \ldots, K$.

Given K models with predictive means $\mu_k(x)$ and variances $\sigma_k^2(x)$, the ensemble predictive moments are:

$$\mu_{\mathrm{DE}}(x) = \frac{1}{K}\sum_{k=1}^{K} \mu_k(x), \qquad \sigma^2_{\mathrm{DE}}(x) = \frac{1}{K}\sum_{k=1}^{K} \big(\sigma_k^2(x) + \mu_k^2(x)\big) - \mu^2_{\mathrm{DE}}(x). \tag{2}$$

The std term in equation 2 follows from the law of total variance and decomposes into:

- Within-model uncertainty: $\frac{1}{K}\sum_k \sigma_k^2(x)$ (aleatoric + epistemic from each BNN),
- Across-model uncertainty: $\frac{1}{K}\sum_k \mu_k^2(x) - \mu_{\mathrm{DE}}^2(x)$ (disagreement between models).

Deep ensembles are simple, embarrassingly parallel. Unlike ensembles of deterministic networks, it inherits the within-model uncertainty quantification of each BNN component. Nevertheless, uniform weighting ignores predictive performance, poorly performing models receive the same weight as strong ones.

2.3 Method 2: Pseudo-Bayesian Model Averaging (Pseudo-BMA)

Pseudo-BMA attempts to rescue the BMA framework in M-open settings by replacing marginal likelihoods with ELPD estimates (cf. Appendix A). The weights are defined as: $w_k = \frac{\exp\left(\widehat{\mathrm{elpd}}_k^{\mathrm{loo}}\right)}{\sum_{j=1}^K \exp\left(\widehat{\mathrm{elpd}}_j^{\mathrm{loo}}\right)}$.
However, the ELPD estimates have non-negligible variance, which can cause instability. To mitigate this, the authors employ regularized weights :

$$w_k = \frac{\exp\left(\widehat{\mathrm{elpd}}_k^{\mathrm{reg}}\right)}{\sum_{j=1}^K \exp\left(\widehat{\mathrm{elpd}}_j^{\mathrm{reg}}\right)},$$

where the regularized ELPD penalizes high-variance estimates (see Equation (6) in Appendix A).

Despite regularization, Pseudo-BMA remains brittle in practice. When one model dominates slightly in $\widehat{\mathrm{elpd}}^{\mathrm{loo}}$, the softmax in Equation (2.3) can still collapse onto that model, negating the benefits of averaging.

2.4 Method 3: Stacking

Stacking takes a fundamentally different approach: rather than deriving weights from a probabilistic model, it directly optimizes the combined predictive performance. The stacking weights solve:

$$\begin{aligned} \max_{\mathbf{w}} \quad & \frac{1}{N}\sum_{n=1}^N \log\left(\sum_{k=1}^K w_k\, \hat{p}_k(y_n \mid x_n, \mathcal{D}_{-n})\right) \\ \text{subject to} \quad & \sum_{k=1}^K w_k = 1, \\ & \forall 1 \le k \le K, w_k \ge 0. \end{aligned} \tag{3}$$

where $\hat{p}_k(y_n \mid x_n, \mathcal{D}_{-n})$ is the LOO predictive density for model k, estimated via PSIS (see Appendix A).

The objective in Equation (3) is concave in $\mathbf{w}$ (since $\log$ is concave and the argument is linear in $\mathbf{w}$), so standard convex optimization algorithms efficiently find the global optimum.

The key distinction with Pseudo-BMA is that stacking optimizes the mixture's ELPD directly, whereas Pseudo-BMA evaluates models independently then assigns weights. This allows stacking to leverage model complementarity: a model with poor individual ELPD can still receive weight if it improves the ensemble. Mathematically:

- Pseudo-BMA: $w_k \propto \exp(\widehat{\mathrm{elpd}}_k^{\mathrm{loo}})$; weights depend only on individual scores.
- Stacking: $\mathbf{w}^*$ maximizes the *joint* objective; weights depend on how models interact.

2.5 Summary

Table 1 summarizes the three combination methods. The experimental comparison results are discussed in subsection 3.4.

Table 1: Comparison of model combination methods.

Method	Weight definition	Optimizes mixture?	Stability
Deep Ensembles	$w_k = 1/K$ (uniform)	No	High
Pseudo-BMA	$w_k \propto \exp(\widehat{\mathrm{elpd}}_k^{\mathrm{reg}})$	No	Low
Stacking	$\mathbf{w}^* = \mathrm{argmax}_{\mathbf{w}} \mathrm{ELPD}_{\mathrm{mix}}$	Yes	High

3 Experiments

3.1 Summary of Paper Findings

All experiments in Sheinkman and Wade [2025a] are conducted on a synthetic one-dimensional regression task

$$x \sim \mathrm{Unif}([0, 2]), \quad y = \sin(10x)\,\frac{x}{2} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.25),$$

with $N = 500$ training observations and $\tilde{N} = 100$ test observations, complemented by a real-data evaluation on the NASA Langley Glide-Back Booster (LGBB) dataset. Performance is assessed through RMSE, empirical coverage (EC) of the 95% credible interval, and inference time (TT).

The paper's central finding is that **mfVIR achieves the best practical trade-off between accuracy, calibration, and computational cost.** Sigmoid activation degenerates under mean-field VI at large widths, while HMC, though more accurate, severely underestimates uncertainty and becomes computationally prohibitive with depth. In OOD settings, HMC coverage approaches zero while mfVIR maintains sensible uncertainty estimates. Among model averaging strategies, stacking and deep ensembles consistently outperform pseudo-BMA, which collapses onto a single model.

3.2 Reproduction

Starting from the authors' repository Sheinkman and Wade [2025b], which we refactored for clarity and modularity, we reimplemented the width and depth sweeps for the mfVI experiments in this repository: `https://github.com/lucasmbll/BNN_architecture_choice`.

Width sweep. Our results confirm the paper's main findings. The sigmoid activation fails across all widths regardless of prior, with posterior collapse to the prior. For ReLU networks, accuracy improves with moderate width increases, consistent with the paper's claims. We note one discrepancy: at very large width ($D_1 = 2000$), accuracy degrades relative to intermediate width under the Gaussian prior, suggesting a non-monotonic effect of overparameterisation that the paper does not highlight.

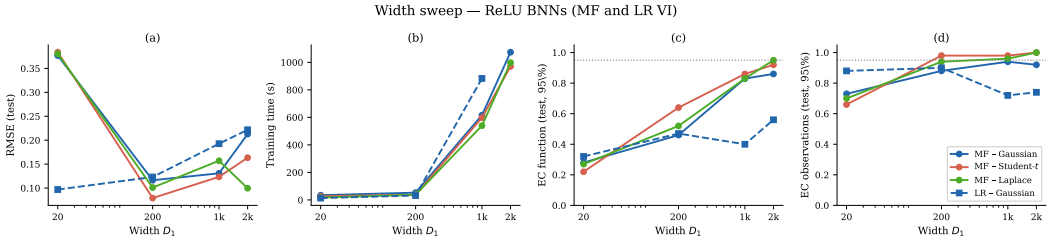

Figure 1: Width sweep reproduction: RMSE and EC as a function of hidden units D_1 for mfVIR with Gaussian, Student-t, and Laplace priors, and LR-VIR with Gaussian prior.

Depth sweep. For shallow networks, mfVIR accuracy and coverage improve with depth, reproducing the paper's finding that depth compensates for the mean-field restriction. Beyond $L = 4$, however, we observe a sharp posterior collapse that the paper reports only mildly. We attribute this discrepancy to differences in initialisation strategy between implementations.

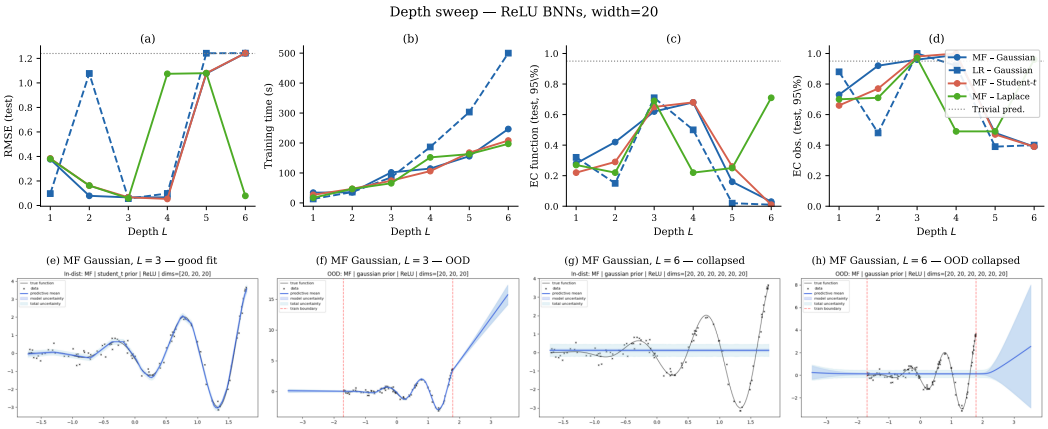

Figure 2: Depth sweep reproduction: RMSE and EC as a function of depth L for mfVIR and LR-VIR with Gaussian prior. The sharp degradation beyond $L = 4$ is more pronounced than reported in the original paper.

3.3 Extensions

We introduce two extensions evaluated on the same experimental grid: a **Laplace prior** as a sparsity-inducing alternative to the Gaussian and Student-t priors of the original work, and a **low-rank Gaussian variational family** (LR-VI), which introduces limited posterior correlations through a rank-r perturbation of the diagonal covariance, as motivated in Section 1.2.

Effect of prior choice. Prior choice has no meaningful effect on sigmoid networks under any configuration, reinforcing the conclusion that activation function is the dominant bottleneck. For ReLU networks, the picture is more nuanced. At intermediate width, Student-t priors yield better accuracy than Gaussian, consistent with the paper's expectation but more clearly demonstrated here. At large width, the Laplace prior substantially recovers the accuracy lost under Gaussian, suggesting that sparsity regularisation becomes important in the overparameterised regime. However, the high coverage achieved by the Laplace prior at large width is accompanied by wide credible intervals, indicating conservative rather than well-calibrated uncertainty. Overall, prior choice matters more at scale than the paper's analysis suggests.

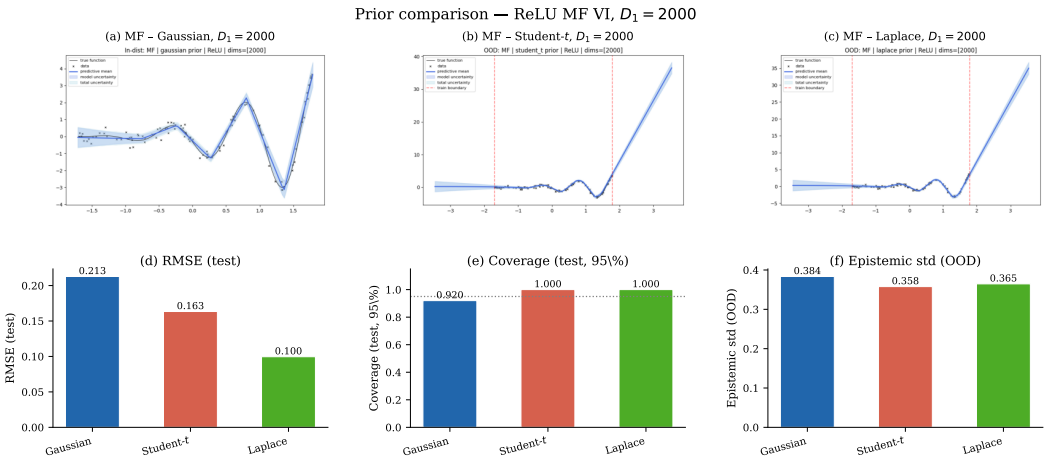

Figure 3: Effect of prior choice on mfVIR: predictive mean and uncertainty bands under Gaussian, Student-t, and Laplace priors at representative widths. The Laplace prior at large width yields wider credible intervals, reflecting conservative rather than well-calibrated uncertainty.

Low-rank variational inference. For shallow and medium-width networks, LR-VI and MF-VI perform comparably in both accuracy and coverage, with no consistent gain from the richer posterior family. For deep networks ($L \geq 4$), LR-VI collapses similarly to MF-VI, confirming that low-rank posterior correlations do not resolve the collapse observed in narrow deep networks. The key practical finding is a scalability constraint at large width: LR-VI with Gaussian prior becomes roughly $20\times$ slower than MF-VI, rendering it impractical. Combining LR-VI with the Laplace prior substantially reduces this overhead while preserving competitive accuracy and coverage, offering the most favourable trade-off among LR configurations.

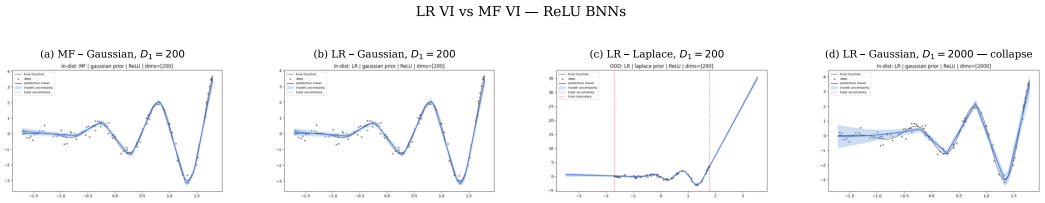

Figure 4: Low-rank VI comparison: predictive mean and uncertainty bands for mfVIR-Gaussian, LR-VIR-Gaussian, and LR-VIR-Laplace at representative widths, illustrating comparable performance at moderate width and the collapse or scalability failure at large width.

Table 2 summarises the key outcomes of our extension experiments.

Table 2: Key extension results on the synthetic 1D regression task. Relative time is normalised to MF-Gaussian at each width. Entries marked – are omitted for conciseness.

Width	Family	Prior	RMSE	EC	Rel. Time
$D_1 = 200$	MF	Gaussian	0.116	0.88	$1\times$
$D_1 = 200$	MF	Student-t	0.079	–	–
$D_1 = 200$	LR	Gaussian	0.123	0.90	–
$D_1 = 2000$	MF	Gaussian	0.213	–	$1\times$
$D_1 = 2000$	MF	Laplace	0.100	1.00	–
$D_1 = 2000$	LR	Gaussian	–	–	$\sim 20\times$
$D_1 = 2000$	LR	Laplace	0.155	0.99	$\sim 2\times$

3.4 Averaging and stacking methods

The authors compare deep ensembles, pseudo-BMA, and stacking on $K = 10$ mfVIR20 models (mean field VI with ReLU activation) in two OOD scenarios: *complement-distributions* (disjoint train/test regions) and *related-distributions* (test data from a slightly broader domain).

Paper findings. Pseudo-BMA performs worst, collapsing onto a single model and producing overconfident predictions. Deep ensembles provide stable uncertainty estimates. Stacking yields marginal accuracy gains and better-calibrated intervals, especially at domain boundaries.

Weight distribution analysis. We reproduced the experiment on the synthetic dataset to visualize the weight distributions across models. Figures 5 and 6 show the individual ELPD scores and resulting weights for each method.

We remark that Pseudo-BMA collapse is severe in both cases due to the softmax amplifying small ELPD differences. However stacking exploits complementarity model 5 receives highest weight in the complement task despite a mediocre individual ELPD.

4 Critical Evaluation

The paper delivers an empirical comparison demonstrating that the optimal pairing of architecture and inference is context dependent. Its contributions are however primarily empirical: the findings

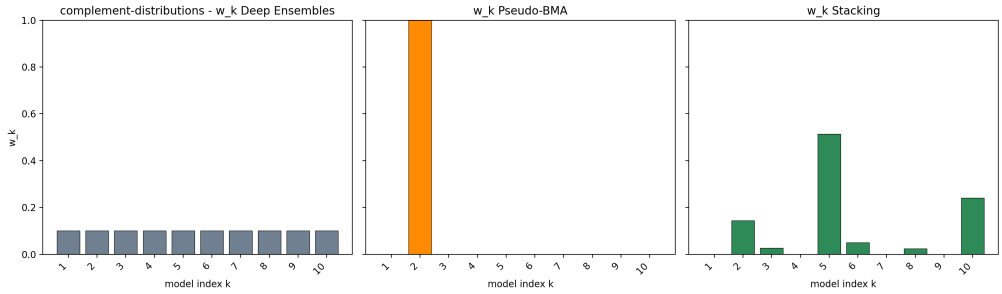

Figure 5: Complement-distributions task: Pseudo-BMA collapses onto model 2 ($w_2 \approx 1$), while stacking distributes weight across models 2, 5, 6, and 10.

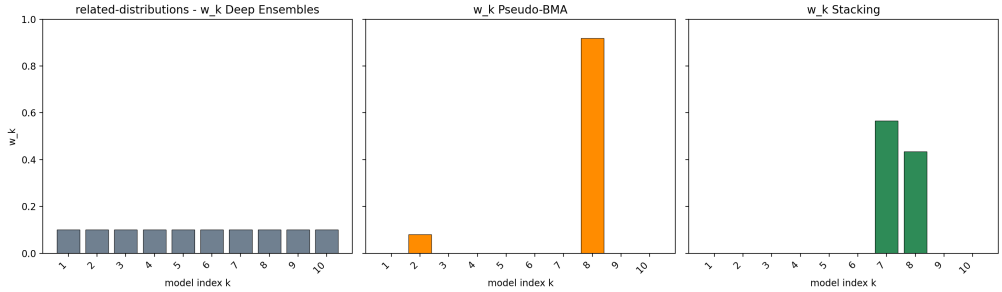

Figure 6: Related-distributions task: Pseudo-BMA concentrates most of the weight on model 8. Stacking selects only models 7 and 8.

characterise BNN behaviour under specific choices but offer no formal guarantees. This limitation is significant given the trajectory of modern deep learning, where state-of-the-art models operate at depths and widths several orders of magnitude beyond those studied here. For such architectures, Bayesian inference remains largely challenging with current methods, and the conclusions drawn from shallow low-width networks cannot be extrapolated with confidence.

Two further gaps are worth noting. First, the evaluation relies on low-dimensional synthetic data and a single tabular physics dataset, leaving it unproven whether the reported dynamics translate to high-dimensional structured data such as images or sequences, where deep learning is most prominently applied.

Our reproduction and extensions reinforce these concerns. The sensitivity of inference quality to activation choice, and the posterior collapse observed under both MF and LR-VI for deep networks, suggest that the practical reliability of variational BNNs is more fragile than the paper's framing implies. Our low-rank extension further shows that richer variational families do not automatically improve performance and introduce substantial scalability costs, underlining the need for approximations that are simultaneously expressive and tractable.

References

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015.

Alisa Sheinkman and Sara Wade. Understanding the trade-offs in accuracy and uncertainty quantification: Architecture and inference choices in bayesian neural networks. *arXiv preprint arXiv:2503.11808*, 2025a.

Alisa Sheinkman and Sara Wade. Bayesian neural networks (bnns) examples". `https://github.com/sheinkmana/ArchitectureofBNNs`, 2025b. GitHub repository.

A Expected Log Pointwise Predictive Density (ELPD)

The model combination methods in Section 2 rely on estimating the **expected log pointwise predictive density** (ELPD), which measures how well a model predicts held-out data. For a model M_k, the ELPD is defined as:

$$\mathrm{elpd}_k = \sum_{n=1}^{N} \mathbb{E}_{p^*} \left[\log p(y_n \mid x_n, \mathcal{D}_{-n}, M_k) \right], \tag{4}$$

where $\mathcal{D}_{-n} = \mathcal{D} \setminus \{(x_n, y_n)\}$ denotes the leave-one-out (LOO) training set, and the expectation is over the true data-generating distribution p^*.

A.1 PSIS-LOO Estimation

Since p^* is unknown and exact LOO cross-validation requires N refits, we approximate ELPD using **Pareto-Smoothed Importance Sampling LOO** (PSIS-LOO). The key idea is to reweight posterior samples from the full-data posterior to approximate the LOO posterior.

Given posterior samples $\{\theta_k^{(s)}\}_{s=1}^{S}$ from $p(\theta_k \mid \mathcal{D}, M_k)$, the raw importance weights for observation n are:

$$r_n^{(s)} = \frac{1}{p(y_n \mid x_n, \theta_k^{(s)}, M_k)}. \tag{5}$$

These weights can have high variance when some observations are poorly predicted. To stabilize the estimator, PSIS fits a generalized Pareto distribution to the upper tail of the weights, yielding *smoothed weights* $\tilde{r}_n^{(s)}$.

The PSIS-LOO estimate of ELPD is then: $\widehat{\mathrm{elpd}}_k^{\mathrm{loo}} = \sum_{n=1}^{N} \widehat{\mathrm{elpd}}_{k,n}^{\mathrm{loo}}$, where the pointwise contribution is $\widehat{\mathrm{elpd}}_{k,n}^{\mathrm{loo}} = \log \left(\frac{\sum_{s=1}^{S} \tilde{r}_n^{(s)} \, p(y_n \mid x_n, \theta_k^{(s)}, M_k)}{\sum_{s=1}^{S} \tilde{r}_n^{(s)}} \right)$.

A.2 Regularized ELPD for Pseudo-BMA

The ELPD estimates have finite-sample variance. For Pseudo-BMA, the authors use a regularized version that penalizes high-variance estimates:

$$\widehat{\mathrm{elpd}}_k^{\mathrm{reg}} = \widehat{\mathrm{elpd}}_k^{\mathrm{loo}} - \frac{1}{2} \widehat{\mathrm{se}}_k, \tag{6}$$

where the standard error is estimated from the pointwise contributions:

$$\widehat{\mathrm{se}}_k = \sqrt{N \cdot \widehat{\mathrm{Var}} \left(\widehat{\mathrm{elpd}}_{k,n}^{\mathrm{loo}} \right)} = \sqrt{\sum_{n=1}^{N} \left(\widehat{\mathrm{elpd}}_{k,n}^{\mathrm{loo}} - \frac{\widehat{\mathrm{elpd}}_k^{\mathrm{loo}}}{N} \right)^2}. \tag{7}$$

This regularization shrinks the weights toward uniformity when ELPD estimates are noisy, providing a form of Bayesian regularization motivated by a log-normal approximation to the sampling distribution of $\widehat{\mathrm{elpd}}_k^{\mathrm{loo}}$.